

โครงข่ายอัจฉริยะแห่งอนาคต

บทพิสูจน์เชิงประจักษ์ของ HEAIN Core Protocol: สมองกลอัจฉริยะสำหรับ Edge AI ที่คงทน ยืดหยุ่น ขยายตัวไร้ขีดจำกัด ปลอดภัย และเคารพอธิปไตยข้อมูล

จักรกรกษ บุญมา (Jakkrit Boonma)

นักวิจัยอิสระ · jaggid.boonma@yahoo.co.th · +66 081-007-4833

วันที่: 29 กันยายน 2569 (2026)

การพัฒนา: heain-core (Go 1.22), HP Z840 Workstation

Repository: github.com/JakkritB/heain-core

บทคัดย่อ

สถาปัตยกรรม Hybrid Edge AI Node [1][2] ได้ถูกนำเสนอขึ้นในฐานะกรอบแนวคิดเชิงทฤษฎีเพื่อแก้ไขปัญหาความหน่วงของคลาวด์และอธิปไตยข้อมูล ผ่านโครงสร้างการกำกับดูแลแบบ Fractal 4 เลเยอร์ และ 7 อัลกอริทึมหลัก แม้ว่าข้อกำหนดเชิงบรรทัดฐานใน HEAIN Core Protocol (HEAIN-CORE-1.0) [3][4] จะช่วยวางกฎเกณฑ์และรูปแบบการสื่อสารในระดับมาตรฐานแล้ว แต่เอกสารทั้งสองฉบับยังคงเป็นเพียงกรอบแนวคิดและข้อกำหนดทางทฤษฎีที่ยังไม่มีการพิสูจน์ผลลัพธ์บนฮาร์ดแวร์จริง บทความนี้นำเสนอการพัฒนาและการประเมินผลเชิงประจักษ์ของ HEAIN-CORE-1.1 ซึ่งเป็นการต่อยอดจาก HEAIN-CORE-1.0 ด้วยการเขียนโค้ดจริงด้วยภาษา Go บนเวิร์กสเตชัน HP Z840 โดยมุ่งเน้นการตรวจสอบการทำงานของอัลกอริทึมหลักทั้ง 7 (P1-P7) ภายใต้การรับส่งข้อมูลผ่านเครือข่าย mTLS และ HTTP จริง ผลการศึกษาไม่เพียงแต่ยืนยันว่าโปรโตคอลดังกล่าวสามารถทำงานได้จริงบนฮาร์ดแวร์กายภาพเท่านั้น แต่ยังค้นพบและเติมเต็ม 4 กลไกส่วนขยายที่จำเป็นต่อการใช้งานในระบบอุตสาหกรรม ได้แก่ Duty Profile, Role Ceiling, Escalation Saturation และ P2 Candidate-Pool Discovery งานวิจัยนี้จึงทำหน้าที่เป็นสะพานเชื่อมสำคัญที่พิสูจน์ว่า HEAIN Core Protocol สามารถขับเคลื่อนโครงข่าย Edge AI ให้มีความคงทน ยืดหยุ่น ขยายตัวอย่างไร้ขีดจำกัด ปลอดภัยด้วยแนวคิด Zero-Trust และเคารพอธิปไตยข้อมูลได้อย่างสมบูรณ์ในทางปฏิบัติ

คำสำคัญ: HEAIN Core Protocol, Hybrid Edge AI Node, Empirical Implementation, Fractal Governance, Escalation Saturation, Live Verification, Edge Computing

1. บทนำ

1.1 ที่มาและแรงจูงใจ

Hybrid Edge AI Node [1][2] เป็นบทความสถาปัตยกรรมที่รวมงานวิจัยหลายชิ้นที่แยกจากกันเดิม (hierarchical edge orchestration, hierarchical federated aggregation, swarm intelligence, distributed leader election, human-in-the-loop AI governance และ sovereign edge computing) ให้เป็นสถาปัตยกรรมเดียวที่ปรับขนาดตามองค์กรได้ บทความนี้ตอบคำถามว่า "ควร" ออกแบบระบบอย่างไร แต่ยังไม่ตอบคำถามว่า "ต้องสร้างอย่างไร" ให้ทำงานร่วมกันได้จริงในระดับสาย (wire level) HEAIN Core Protocol (HEAIN-CORE-1.0) [3][4] จึงถูกเขียนขึ้นเป็นข้อกำหนดคู่ขนาน (companion specification) ในสเปค RFC 2119 [5] เพื่อปิดช่องว่างนี้ โดยกำหนดรูปแบบ Envelope, การกำหนดตำแหน่งโหนดแบบ heain://, และกฎการปกครองของกระบวนการหลักทั้งเจ็ด อย่างไรก็ตาม HEAIN Core Protocol [3][4] เองก็ยังเป็นเพียงข้อกำหนดที่ยังไม่มีการ implement อ้างอิงที่สมบูรณ์ (reference implementation ที่มีอยู่ครอบคลุมเพียงกระบวนการที่ 1-3 บางส่วนเท่านั้น ตามที่ระบุไว้ในบทที่ 9 ของบทความนั้น) โครงการ heain-core จึงเริ่มขึ้นเพื่อสร้าง implementation ที่ครบทั้งเจ็ดกระบวนการ และพิสูจน์ด้วยการทดสอบจริงบนฮาร์ดแวร์ ไม่ใช่เพียง pseudocode หรือ unit test

1.2 ช่องว่างที่พบระหว่างการ Implement

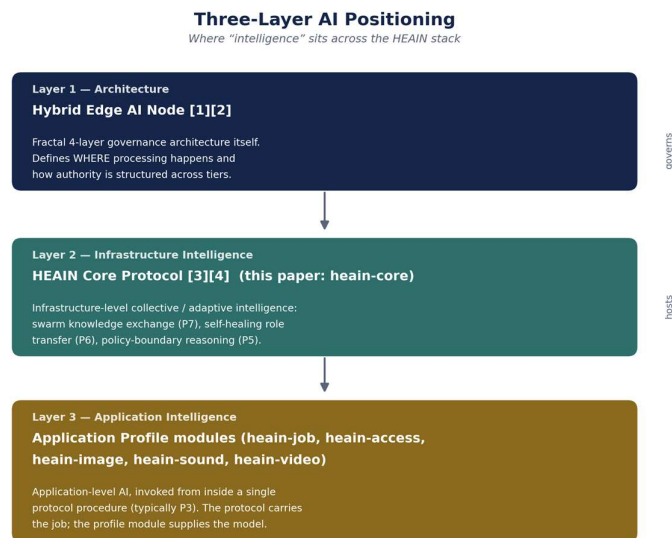
ก่อนเริ่มพัฒนา ได้ตรวจสอบช่องว่างอย่างเป็นระบบระหว่าง Hybrid Edge AI Node [1][2] และ HEAIN Core Protocol [3][4] พบว่ามีกลไกอย่างน้อยหกจุดที่บทความกล่าวถึงหรือสื่อถึงความจำเป็น แต่บทความที่สองไม่ได้แปลงเป็นข้อกำหนดที่ implement ได้ การพัฒนา heain-core จึงดำเนินไปโดยยึดรายการช่องว่างนี้เป็นแนวทาง และตัดสินใจเชิงขอบเขตอย่างชัดเจนทุกจุดที่ขอบเขตของเวอร์ชันนี้ ถูกลดทอนจากแบบที่บทความทั้งสองอาจสื่อถึง เวอร์ชันนี้ของ heain-core ครอบคลุมเฉพาะกลไกที่ถูกเขียนโค้ดจริงและทดสอบยืนยันบนฮาร์ดแวร์จริงแล้ว (live-verified) ส่วนกลไกใดที่ยังไม่ implement หรือ implement เพียงบางส่วน จะถูกระบุไว้อย่างชัดเจนในแต่ละหัวข้อว่าเป็นขอบเขตที่สนใจเลื่อนออกไปสำหรับเวอร์ชันถัดไป ไม่ใช่ช่องว่างที่ถูกมองข้ามโดยไม่รู้ตัว

1.3 ขอบเขตของเอกสารนี้

เอกสารฉบับนี้ครอบคลุมเฉพาะ Layer 2 (HEAIN Core Protocol [3][4]) ไม่รวม Layer 3 (โมดูลประยุกต์เฉพาะทาง เช่น heain-job) ซึ่งยังอยู่ในขั้นวางแผน เนื้อหาส่วนที่เหลือนำเสนอสถาปัตยกรรมเชิงบัญญัติ อัลกอริทึมหลักทั้งเจ็ดพร้อมสถานะการ implement จริง กลไกเสริมที่ค้นพบระหว่างการพัฒนา ระเบียบวิธีตรวจสอบ ผลลัพธ์โดยรวม ข้อจำกัด และแผนงานในอนาคต

1.4 ตำแหน่งของปัญญาประดิษฐ์สามชั้น (Three-Layer AI Positioning)

เพื่อไม่ให้เกิดความสับสนว่า "ความฉลาด" ของระบบอยู่ตรงไหน จึงแยกตำแหน่งของ AI ออกเป็นสามชั้นอย่างชัดเจน ชั้นที่หนึ่งคือ Hybrid Edge AI Node [1][2] ซึ่งเป็นตัวสถาปัตยกรรมเอง กำหนดว่าการประมวลผล "เกิดที่ไหน" และอำนาจถูกจัดโครงสร้างอย่างไรข้ามชั้น ชั้นที่สองคือ HEAIN Core Protocol [3][4] ซึ่งเอกสารฉบับนี้รายงานผล เป็นปัญญาระดับโครงสร้างพื้นฐาน (infrastructure-level intelligence) ได้แก่การแลกเปลี่ยนองค์ความรู้ข้ามโหนดแบบ swarm (P7) การสืบทอดบทบาทแบบ self-healing (P6) และการให้เหตุผลเชิงขอบเขตนโยบาย (P5) ส่วนชั้นที่สามคือปัญญาระดับแอปพลิเคชัน (application-level AI) ซึ่งอยู่ในโมดูล Application Profile เช่น heain-job, heain-access, heain-image, heain-sound และ heain-video โดยแต่ละโมดูลถูกเรียกใช้งานจากภายในกระบวนการเดียวของโปรโตคอล (โดยทั่วไปคือ P3 — Secure Execution & Staging) กล่าวคือโปรโตคอลทำหน้าที่ลำเลียงงาน ส่วนโมดูล Application Profile เป็นผู้ให้โมเดลการแข่งขันนี้อยู่นอกขอบเขตของเอกสารฉบับนี้ในแง่การ implement (ดู 1.3) แต่จำเป็นต้องระบุไว้เพื่อกำหนดตำแหน่งของ heain-core ให้ชัดเจนภายในภาพรวมทั้งระบบ



ภาพที่ 1 — ตำแหน่งของปัญญาประดิษฐ์สามชั้นในสถาปัตยกรรม HEAIN

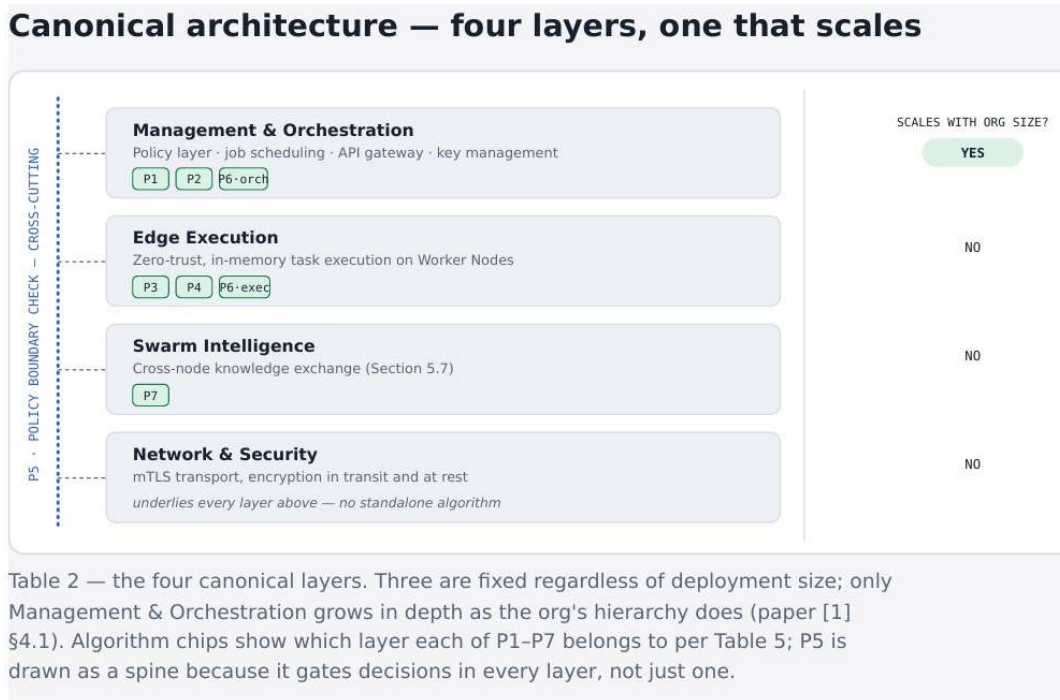
2. สถาปัตยกรรมเชิงบัญญัติ

สถาปัตยกรรมของ Hybrid Edge AI Node [1][2] ถูกกำหนดด้วยสี่ชั้น โดยสามชั้นคงที่และหนึ่งชั้นปรับขนาดตามองค์กร (ตารางที่ 1 — สอดคล้องกับตารางที่ 2 ของ [1][2]):

ชั้น (Layer)	หน้าที่	ปรับขนาดตามองค์กร?
Management & Orchestration	รองรับ Policy layer, การจัดคิวงาน, API gateway, การจัดการกฎและความปลอดภัย	ใช่ — ดู 2.1
Edge Execution	การประมวลผลงานแบบ Zero-Trust ในหน่วยความจำบน Worker Node	ไม่
Swarm Intelligence	การแลกเปลี่ยนองค์ความรู้ข้ามโหนด	ไม่
Network & Security	การขนส่งผ่าน mTLS, การเข้ารหัสทั้งระหว่างส่งและขณะพัก	ไม่

2.1 การขยายทั่วไปข้ามระดับชั้น (N-tier Generalization)

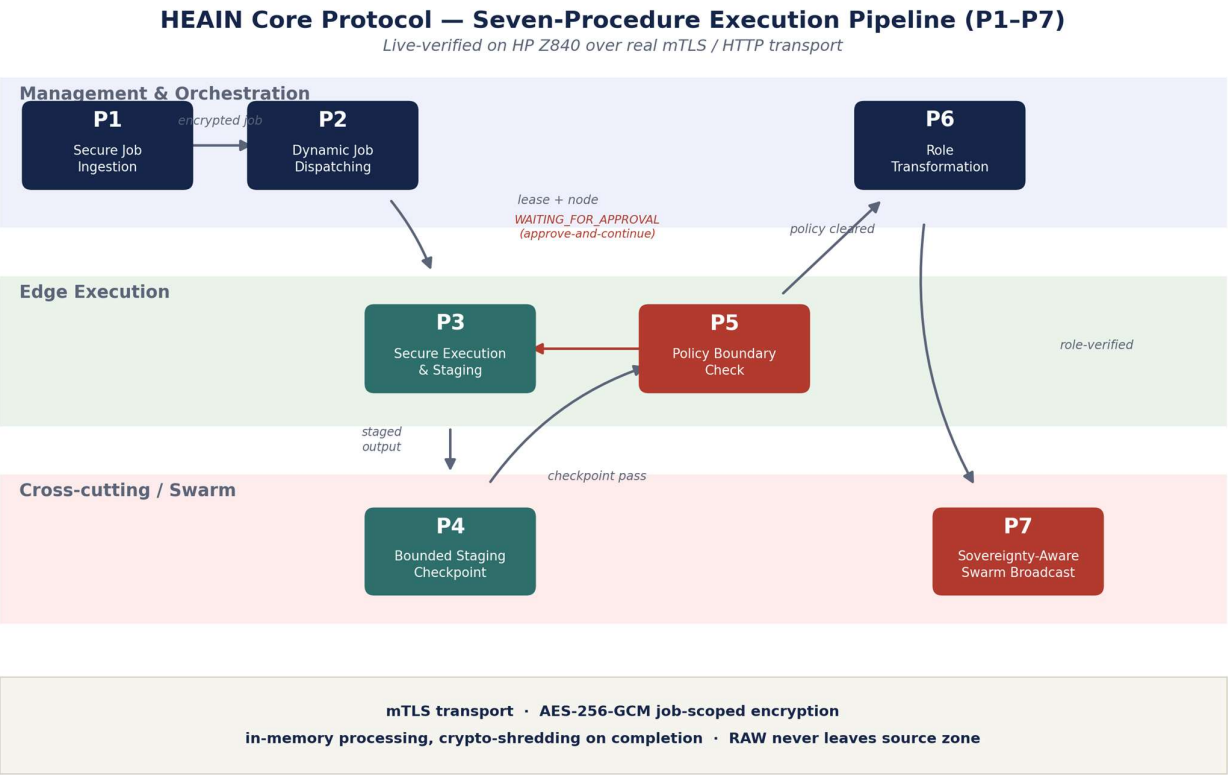
การ implement จริงพบว่าการกำหนด Tier เป็นค่าคงที่สักระดับตายตัวตามที่ HEAIN Core Protocol [3][4] ระบุไว้ (Worker, Zone, Secondary, Global Primary) จำกัดเกินไปสำหรับองค์กรที่มีโครงสร้างต่างกัน จึงตัดสินใจปรับ Tier ให้เป็น free-form descriptive label แทนค่า enum ตายตัว โดยย้ายการตัดสินใจคุณสมบัติผู้สมัครเลื่อนตำแหน่งไปไว้ที่ชั้นของผู้เรียกเอง แทนที่จะบังคับด้วยการเทียบชื่อ tier ในโค้ดกลาง ตัดสินใจและบันทึกไว้ใน design-notes/n-tier-generalization.md [8] การเปลี่ยนแปลงนี้ implement ในแพ็คเกจ internal/protocol/roletransform และผ่านการ build/test เรียบร้อยแล้ว (commit efa30d5)



ภาพที่ 2 — สถาปัตยกรรมเชิงบัญญัติสี่ชั้นและสถานะการ build จริงของ heain-core

3. อัลกอริทึมหลักทั้งเจ็ด (P1-P7)

ภาพที่ 3 สรุปกลไกการทำงานของอัลกอริทึมหลักทั้งเจ็ดและเส้นทางข้อมูลระหว่างกัน เป็นภาพหลักที่ใช้อ้างอิงได้ทั้งในเอกสารนี้ และการนำเสนอทางวิชาการ/ธุรกิจ:



ภาพที่ 3 (ภาพหลัก) — กลไกการทำงานของ HEAIN Core Protocol ตลอดกระบวนการ P1-P7

ตารางที่ 2 สรุปอัลกอริทึมหลักทั้งเจ็ดตามที่ Hybrid Edge AI Node [1][2] ระบุ (สอดคล้องกับตารางที่ 5 ของ [1][2]) และตามที่ HEAIN Core Protocol [3][4] แปลงเป็นกระบวนการ HEAIN-P1 ถึง P7:

#	อัลกอริทึม	ชั้น (Layer)
P1	Secure Job Ingestion	Management & Orchestration
P2	Dynamic Job Dispatching	Management & Orchestration
P3	Secure Execution & Staging	Edge Execution
P4	Bounded Staging Checkpoint	Edge Execution (background)
P5	Policy Boundary Check	Cross-cutting
P6	Role Transformation	Orchestration + Edge Execution
P7	Sovereignty-Aware Swarm Broadcast	Swarm Intelligence

3.1 P1 — Secure Job Ingestion

P1 implement ด้วยแพ็คเกจใหม่ internal/protocol/ingestion (ปิดช่องว่างข้อที่ 1 ตามที่ระบุในหัวข้อ 1.2) — ทันทีที่ไฟล์ดิบ (raw payload) ถูก ingest เข้าระบบ P1 จะออกกฎเฉพาะงานจาก KMS แบบ pluggable (AES-256-GCM ผ่าน LocalKMS สำหรับเวอร์ชันนี้) และเข้ารหัส payload ดิบทันที ก่อนสร้าง Envelope และเผยแพร่เข้าสู่ DurableQueue ที่หมุนหลังด้วย BoltDB เชื่อมเข้ากับ endpoint /ingest จริง การทดสอบจริงบนฮาร์ดแวร์ (2569-09-27) ยืนยันว่า plaintext ของ payload ไม่ปรากฏในไฟล์คิวเลยแม้แต่ช่วงสั้น ๆ ข้อจำกัดที่ทราบคือการจำแนกประเภทงานยังเป็นค่าที่ผู้เรียกกำหนดเอง แทนที่จะถูกอนุมานจาก P5 Policy gate โดยอัตโนมัติตามที่ [3][4] อธิบายไว้

3.2 P2 — Dynamic Job Dispatching

P2 implement ด้วย CapacityProbe ที่สอบถามทรัพยากรจริง (RAM/GPU) และ LeaseStore กันงานเข้า การจัดอันดับผู้สมัครเป็นแบบ deterministic แทน AI-assisted scoring live-verified บน mTLS จริงเมื่อ 2569-09-27 ต่อมาขยายเพิ่มกลไกค้นหากลุ่มผู้สมัครอัตโนมัติ (4.4)

3.3 P3 — Secure Execution & Staging

P3 implement ด้วย SecureExecutor — ถอดรหัส→execute→wipe→route ผ่าน endpoint /execute จริง ใช้ passthroughExecutor แทนตรรกะ Layer 3 ซึ่งยังไม่อยู่จริง live-verified ทั้งสองเส้นทางการส่งมอบเมื่อ 2569-09-27

3.4 P4 — Bounded Staging Checkpoint

P4 implement ด้วย BoundedStaging — กำจัดข้อมูลด้วย crypto-shredding ผ่าน KMS ตรวจสอบ legal hold และระยะเก็บรักษาขั้นต่ำเสมอ พร้อม Sweep() สำหรับย้ายชั้นข้อมูลและยกระดับ escalation โดยระยะเวลาทั้งสองกำหนดได้ผ่านฟิลด์ HotThreshold/EscalateThreshold ของ BoundedStaging (ไม่ได้ตายตัว) ค่าเริ่มต้นที่ใช้ในการทดสอบจริงคือ 48 ชั่วโมงสำหรับย้ายชั้นข้อมูลและ 30 วันสำหรับยกระดับ ระหว่างทดสอบจริงพบและแก้ไข idempotency bug สองครั้ง

3.5 P5 — Policy Boundary Check

P5 implement เส้นทางด่วนสามแบบและ WAITING_FOR_APPROVAL ช่องว่างสำคัญที่พบคือไม่มี completion path หลังได้รับอนุมัติ แก่ด้วยกลไก approve-and-continue live-verified ครบทั้งฝั่ง P7 (2569-09-27) และฝั่ง P4 ผ่าน pipeline เต็ม P1→P2→P3→P4 (2569-09-28)

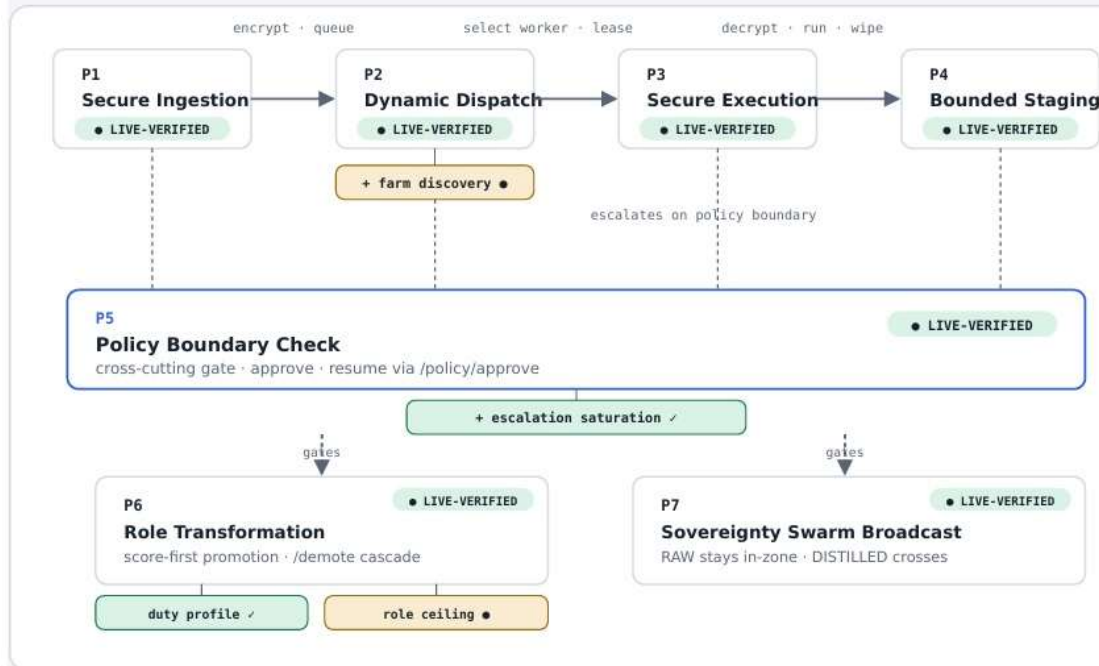
3.6 P6 — Role Transformation

P6 ครอบคลุมการเลื่อนตำแหน่งแบบปลอดภัยจาก split-brain, /demote cascade ทั้งกลุ่ม, acting-role persistence และ Duty Profile ทุกกลไกทดสอบจริงบนฮาร์ดแวร์และพบ/แก้ข้อบกพร่องจริงระหว่างทาง

3.7 P7 — Sovereignty-Aware Swarm Broadcast

P7 รับประกันว่า RAW ไม่ออกนอกโซนต้นทาง มีเพียง DISTILLED เท่านั้นที่ข้ามโซนได้ พร้อม EmbeddingGuard บังคับบล็อก dense vector เชื่อมต่อเครือข่ายจริงและ live-verified แบบ end-to-end เมื่อ 2569-09-27

Core algorithms — job lifecycle, gated by governance



P1→P4 is the job lifecycle; every stage can escalate through P5, which alone can approve and resume a parked action. P6 and P7 are governance/swarm mechanisms P5 also gates. Chips below a box mark extensions built on top of that mechanism: ✓ live-verified, ● build-verified with live test still pending.

● live-verified on real hardware ● build-verified, live functional test still pending

ภาพที่ 4 — pipeline การส่งมอบงานจริงจาก P1 ถึง P4 บนฮาร์ดแวร์ HP Z840

4. กลไกเสริมที่ค้นพบระหว่างการพัฒนา

ระหว่างการ implement อัลกอริทึมหลักทั้งเจ็ด พบความจำเป็นของกลไกเสริมสี่รายการที่ Hybrid Edge AI Node [1][2] กล่าวถึงเพียงในเชิงหลักการ หรือ HEAIN Core Protocol [3][4] ไม่ได้แปลงเป็นข้อกำหนดที่ implement ได้ กลไกทั้งหมดถูกบันทึกการตัดสินใจเชิงขอบเขตไว้อย่างชัดเจนใน design-notes/n-tier-generalization.md [8] และ claude/progress-log.md [9]

4.1 Duty Profile

กลไกจำกัดความสามารถ (capacity-gated) ที่ผูกกับแต่ละ role slot ใน Zone Registry Implement และ live-verified บนฮาร์ดแวร์จริงแล้ว (2569-09-26)

4.2 Promotion Scope / Eligible Role Ceiling

ปิดช่องว่างด้านความปลอดภัยที่ HEAIN-SD-1.0 (ส่วนหนึ่งของ [3][4]) กำหนดไว้ว่าโหนดระดับ EDGE ต้องถูกจำกัดให้อยู่บทบาท WORKER เท่านั้น ลดรูปเป็น same-tier equality check เนื่องจากโค้ดเบสยังไม่มีกลไกเลื่อนตำแหน่งหลายระดับ build-verified แล้ว (commit 4d018a5, 2569-09-28) รอ live functional verification

4.3 Escalation Saturation Governance-Cadence Safeguard

Hybrid Edge AI Node [1][2] ระบุความเสี่ยงว่าผู้ดูแลระบบมีแนวโน้มขยาย threshold อัตโนมัติเพื่อลดภาระงานของตนเอง และเสนอให้ทบทวนตามวงรอบธรรมาภิบาลที่แยกจากการปรับแต่งเชิงปฏิบัติการ แต่ไม่ได้ออกแบบกลไกจริงไว้ และ HEAIN Core Protocol [3][4] ก็ไม่ได้กล่าวถึงเลย heain-core จึงออกแบบและ implement กลไกนี้ขึ้นใหม่ทั้งหมด live-verified ครบทั้งสองขั้นตอนแล้ว (commit 4d018a5, 2569-09-28)

4.4 P2 Candidate-Pool Discovery via Zone Registry

ปิดช่องว่างที่ /dispatch เดิมต้องให้ผู้เรียกขานรายการผู้สมัครทั้งหมดเอง ไม่มีกลไกค้นพบสมาชิกฟาร์ม Worker อัตโนมัติ เวอร์ชันนี้ใช้รายการสถิติที่ผู้ดูแลระบบประกาศไว้ล่วงหน้าเป็นค่าสำรอง build-verified แล้ว (commit 4d018a5, 2569-09-28) รอ live functional verification

BEYOND THE BASE PROTOCOL			
Extensions layered on top of P1-P7			
MECHANISM	ATTACHES TO	WHAT IT DOES	STATUS
Duty Profile	P6	Gates which Application-AI duties a promoted node can serve, based on its real capacity — soft, post-selection degradation.	● live-verified
Promotion Scope / Role Ceiling	P6	Hard, pre-selection exclusion — a node whose ceiling equals its own tier never becomes a promotion candidate (e.g. an Edge Node).	● build-verified, live test pending
Escalation Saturation	P5	Governance-cadence tracker — flags when approval thresholds are widened too often or go unreviewed past a set cadence.	● live-verified
P2 Farm-Candidate Discovery	P2	Static, operator-declared Worker farm list (- farm-worker-addr/-ids) that /dispatch falls back to when a caller omits candidates.	● build-verified, live test pending
heain-core · Go 1.22 · target: HP Z840 workstation · design basis: design-notes/n-tier-generalization.md & claude/progress-log.md, 2026-09-28.			

ภาพที่ 5 — สรุปกลไกเสริมทั้งสี่และสถานะการตรวจสอบจริงบนฮาร์ดแวร์

5. ระเบียบวิธีการพัฒนาและตรวจสอบ

โครงการนี้ยึดมาตรฐานการตรวจสอบสองระดับตลอดการพัฒนา ซึ่งเข้มงวดกว่าที่ [1][2] ยอมรับไว้เองว่ายังเป็นเพียง pseudocode ที่ไม่ผ่านการ benchmark:

- Build-verified — ทุกการเปลี่ยนแปลงต้องผ่าน go build ./..., go vet ./... และ go test ./... บนโค้ดเบสจริงทั้งหมด
- Live-verified — กลไกสำคัญทุกตัวต้องถูกทดสอบจริงผ่านการรันโหนดจริงบนฮาร์ดแวร์ (HP Z840) และเรียกผ่าน HTTP/mTLS จริง

มาตรฐานนี้นำไปสู่การค้นพบข้อบกพร่องจริงหลายจุดที่ unit test ไม่สามารถจับได้ เช่น bug ความไม่เป็น idempotent ของการกำจัดข้อมูลใน P4 และ bug การรายงานสถานะที่ค้างค่าเก่าใน P6

6. ผลลัพธ์โดยรวม

#	อัลกอริทึม	สถานะ	ผลการทดสอบจริง
P1	Secure Job Ingestion	เสร็จสมบูรณ์	plaintext ไม่หลงเหลือในไฟล์คิวจริง
P2	Dynamic Job Dispatching	เสร็จสมบูรณ์	เลือก worker จาก capacity จริง
P3	Secure Execution & Staging	เสร็จสมบูรณ์	execute-and-wipe ทั้งสองเส้นทางผ่านจริง
P4	Bounded Staging Checkpoint	เสร็จสมบูรณ์	crypto-shred จริง, พบ/แก้ idempotency bug 2 ครั้ง
P5	Policy Boundary Check	เสร็จสมบูรณ์	approve-and-continue resume จริง
P6	Role Transformation	เสร็จสมบูรณ์	promotion/demote/duty profile ผ่านจริง
P7	Sovereignty-Aware Swarm Broadcast	เสร็จสมบูรณ์	แยกโซน RAW/DISTILLED ถูกต้อง

7. ข้อจำกัดและงานที่ยังไม่เสร็จ

- AI Reasoning Core ยังไม่ implement — P2 ใช้ deterministic fallback แทนตลอด
- Provisioning flow แบบเต็มรูปแบบยังไม่ implement
- การเลื่อนตำแหน่งแบบหลายระดับต่อเนื่องยังไม่มีในโค้ดเบส — Promotion Scope จึงลดรูปเหลือ same-tier equality check
- Zone Registry ยังเป็น static list ไม่ใช่กลไกลงทะเบียนตนเองแบบพลวัต
- Backup/disaster-recovery ถูก defer ไปยังงานพัฒนาในอนาคตอย่างชัดเจน ยังไม่อยู่ในขอบเขตของ v1 นี้

8. แผนงานในอนาคต

- เริ่มต้น proof-of-concept ของ heain-job (Layer 3)
- ทำ live functional verification ให้ครบสำหรับ Promotion Scope และ P2 Candidate-Pool Discovery
- ออกแบบ Zone Registry แบบพลวัตเต็มรูปแบบ เมื่อมีกลไกอื่นที่ต้องพึ่งพาจริง ๆ

9. สรุป

งานที่นำเสนอยืนยันว่าอัลกอริทึมหลักทั้งเจ็ดที่ Hybrid Edge AI Node [1][2] ออกแบบไว้ในเชิงทฤษฎี และ HEAIN Core Protocol (HEAIN-CORE-1.0) [3][4] แปลงเป็นข้อกำหนดเชิงบรรทัดฐาน สามารถ implement ได้จริงและทำงานถูกต้องบนฮาร์ดแวร์จริงผ่าน HTTP/mTLS พร้อมค้นพบและปิดช่องว่างเชิงกลไกสัจพจน์ที่จำเป็นต่อการใช้งานจริงแต่ไม่ได้ถูก implement ไว้ในเอกสารทั้งสองฉบับ HEAIN-CORE-1.1 จึงทำหน้าที่เป็นหลักฐานเชิงประจักษ์ (empirical evidence) ที่ยืนยันความเป็นไปได้ในการ implement ของ [1][2][3][4] และเสนอกลไกเสริมที่ควรถูกบรรจุเข้าเป็นส่วนหนึ่งของสเปกโพรโทคอลอย่างเป็นทางการในรุ่นถัดไป

เอกสารอ้างอิง

- [1] Boonma, J. "Hybrid Edge AI Node: A Fractal Governance Architecture for Sovereignty-Aware Distributed AI Processing." Zenodo, 2026. DOI: 10.5281/zenodo.22851257
- [2] Boonma, J. "Hybrid Edge AI Node: สถาปัตยกรรมการกำกับดูแลแบบยืดหยุ่นสำหรับการประมวลผล AI แบบกระจายศูนย์ที่เคารพอธิปไตยข้อมูล." Zenodo, 2026. DOI: 10.5281/zenodo.22851587
- [3] Boonma, J. "HEAIN Core Protocol: A Normative Specification for Collective Intelligence in Fractal Edge AI Governance." Zenodo, 2026. DOI: 10.5281/zenodo.22858845
- [4] Boonma, J. "HEAIN Core Protocol: ข้อกำหนดเชิงบรรทัดฐานสำหรับปัญหาร่วมในการกำกับดูแล AI แบบยืดหยุ่นที่ขอบเครือข่าย." Zenodo, 2026. DOI: 10.5281/zenodo.22858994
- [5] Bradner, S. "Key words for use in RFCs to Indicate Requirement Levels." IETF RFC 2119, 1997.
- [6] Ongaro, D., and Ousterhout, J. "In Search of an Understandable Consensus Algorithm (Raft)." USENIX Annual Technical Conference, 2014.
- [7] Boonma, J., and Rujipattanapong, S. "On the Study of the Effectiveness of iSCSI Protocols on the Representative Applications." Proceedings of ITC-CSCC 2007, Busan, Korea, July 8–11, 2007, pp. 979–980.
- [8] Boonma, J. "design-notes/n-tier-generalization.md." heain-core repository (internal design note), 2026.
- [9] Boonma, J. "claude/progress-log.md." heain-core repository (internal implementation and verification log), 2026.

ผู้เขียนและเจ้าของลิขสิทธิ์: นายจักรกฤษ บุญมา (นักวิจัยอิสระ) | ORCID: 0009-0000-7165-3787 |
Email: jaggid.boonma@yahoo.co.th | โทร: +66 081-007-4833

© 2569 จักรกฤษ บุญมา สงวนลิขสิทธิ์ทุกประการ กรรมสิทธิ์และลิขสิทธิ์ในเอกสารนี้เป็นของนาย จักรกฤษ บุญมา นับแต่วันที่เผยแพร่ ตามหลักกฎหมายลิขสิทธิ์ที่ใช้บังคับ